

Digital Nurture 5.0 - Java FSE React (Week-1 Work)

Design Patterns and Principles:

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **SingletonPatternExample**.
2. **Define a Singleton Class:**
 - Create a class named **Logger** that has a private static instance of itself.
 - Ensure the constructor of **Logger** is private.
 - Provide a public static method to get the instance of the **Logger** class.
3. **Implement the Singleton Pattern:**
 - Write code to ensure that the **Logger** class follows the Singleton design pattern.
4. **Test the Singleton Implementation:**
 - Create a test class to verify that only one instance of **Logger** is created and used across the application.

Solution:

CODE:

```
class Logger {  
  
    private static Logger instance;  
  
    private Logger() {  
        System.out.println("Logger Instance Created");  
    }  
  
    public static Logger getInstance() {
```

```

        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}

public class SingletonTest {

    public static void main(String[] args) {

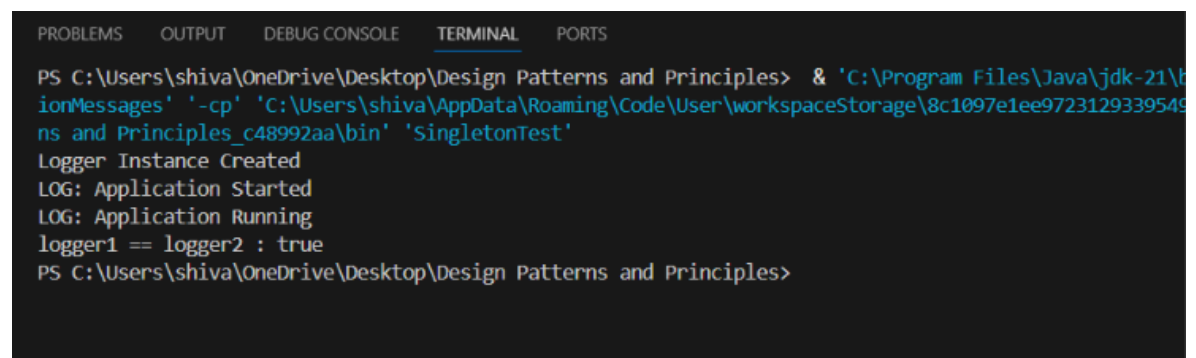
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("Application Started");
        logger2.log("Application Running");

        System.out.println("logger1 == logger2 : " + (logger1 == logger2));
    }
}

```

OUTPUT:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles> & 'C:\Program Files\Java\jdk-21\bin\java.exe' -cp 'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage\8c1097e1ee9723129339549\Design Patterns and Principles_c48992aa\bin' 'SingletonTest'
Logger Instance Created
LOG: Application Started
LOG: Application Running
logger1 == logger2 : true
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles>

```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **FactoryMethodPatternExample**.
2. **Define Document Classes:**
 - Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.
3. **Create Concrete Document Classes:**
 - Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.
4. **Implement the Factory Method:**
 - Create an abstract class **DocumentFactory** with a method **createDocument()**.
 - Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.
5. **Test the Factory Method Implementation:**
 - Create a test class to demonstrate the creation of different document types using the factory method.

Solution:

CODE:

```
interface Document {  
    void open();  
}  
  
class WordDocument implements Document {  
    public void open() {  
        System.out.println("Opening Word Document");  
    }  
}
```

```
class PdfDocument implements Document {  
    public void open() {  
        System.out.println("Opening PDF Document");  
    }  
}
```

```
class ExcelDocument implements Document {  
    public void open() {  
        System.out.println("Opening Excel Document");  
    }  
}
```

```
abstract class DocumentFactory {  
    public abstract Document createDocument();  
}
```

```
class WordDocumentFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}
```

```
class PdfDocumentFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new PdfDocument();  
    }  
}
```

```
class ExcelDocumentFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new ExcelDocument();  
    }  
}
```

```

    }
}

public class FactoryMethodTest {

    public static void main(String[] args) {

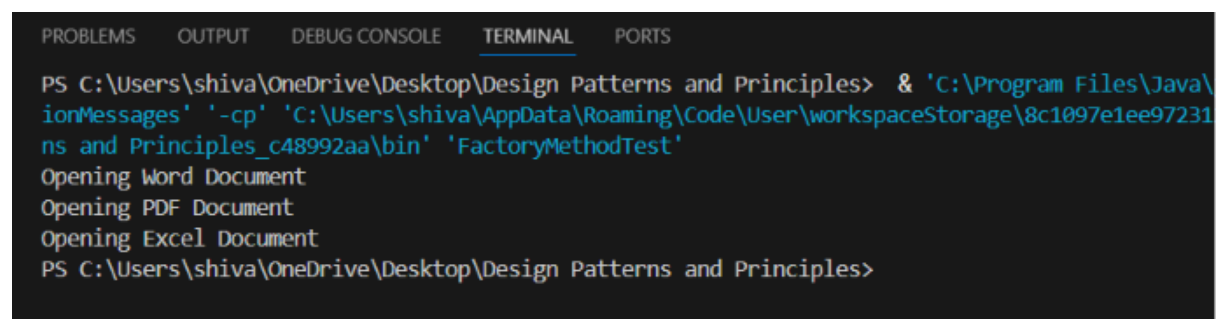
        DocumentFactory wordFactory = new WordDocumentFactory();
        Document wordDoc = wordFactory.createDocument();
        wordDoc.open();

        DocumentFactory pdfFactory = new PdfDocumentFactory();
        Document pdfDoc = pdfFactory.createDocument();
        pdfDoc.open();

        DocumentFactory excelFactory = new ExcelDocumentFactory();
        Document excelDoc = excelFactory.createDocument();
        excelDoc.open();
    }
}

```

OUTPUT:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles> & 'C:\Program Files\Java\
ionMessages' '-cp' 'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage\8c1097e1ee97231
ns and Principles_c48992aa\bin' 'FactoryMethodTest'
Opening Word Document
Opening PDF Document
Opening Excel Document
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles>

```

Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **BuilderPatternExample**.
2. **Define a Product Class:**
 - Create a class **Computer** with attributes like **CPU, RAM, Storage**, etc.
3. **Implement the Builder Class:**
 - Create a static nested Builder class inside Computer with methods to set each attribute.
 - Provide a **build()** method in the Builder class that returns an instance of Computer.
4. **Implement the Builder Pattern:**
 - Ensure that the **Computer** class has a private constructor that takes the **Builder** as a parameter.
5. **Test the Builder Implementation:**

Create a test class to demonstrate the creation of different configurations of Computer using the Builder pattern

Solution:

CODE:

```
class Computer {

    private String cpu;

    private String ram;

    private String storage;

    private String graphicsCard;

    private String operatingSystem;

    private Computer(Builder builder) {

        this.cpu = builder.cpu;
```

```
    this.ram = builder.ram;

    this.storage = builder.storage;

    this.graphicsCard = builder.graphicsCard;

    this.operatingSystem = builder.operatingSystem;
}
```

```
public void display() {
    System.out.println("CPU: " + cpu);
    System.out.println("RAM: " + ram);
    System.out.println("Storage: " + storage);
    System.out.println("Graphics Card: " + graphicsCard);
    System.out.println("Operating System: " + operatingSystem);
    System.out.println();
}
```

```
static class Builder {

    private String cpu;
    private String ram;
    private String storage;
    private String graphicsCard;
    private String operatingSystem;

    public Builder setCPU(String cpu) {
        this.cpu = cpu;
        return this;
    }

    public Builder setRAM(String ram) {
        this.ram = ram;
        return this;
    }
}
```

```
}
```

```
public Builder setStorage(String storage) {  
    this.storage = storage;  
    return this;  
}
```

```
public Builder setGraphicsCard(String graphicsCard) {  
    this.graphicsCard = graphicsCard;  
    return this;  
}
```

```
public Builder setOperatingSystem(String operatingSystem) {  
    this.operatingSystem = operatingSystem;  
    return this;  
}
```

```
public Computer build() {  
    return new Computer(this);  
}  
}  
}
```

```
public class BuilderPatternTest {
```

```
    public static void main(String[] args) {
```

```
        Computer gamingPC = new Computer.Builder()  
            .setCPU("Intel i9")  
            .setRAM("32GB")  
            .setStorage("1TB SSD")
```



```
.setGraphicsCard("NVIDIA RTX 4090")  
.setOperatingSystem("Windows 11")  
.build();
```

```
Computer officePC = new Computer.Builder()
```

```
.setCPU("Intel i5")  
.setRAM("16GB")  
.setStorage("512GB SSD")  
.setOperatingSystem("Windows 10")  
.build();
```

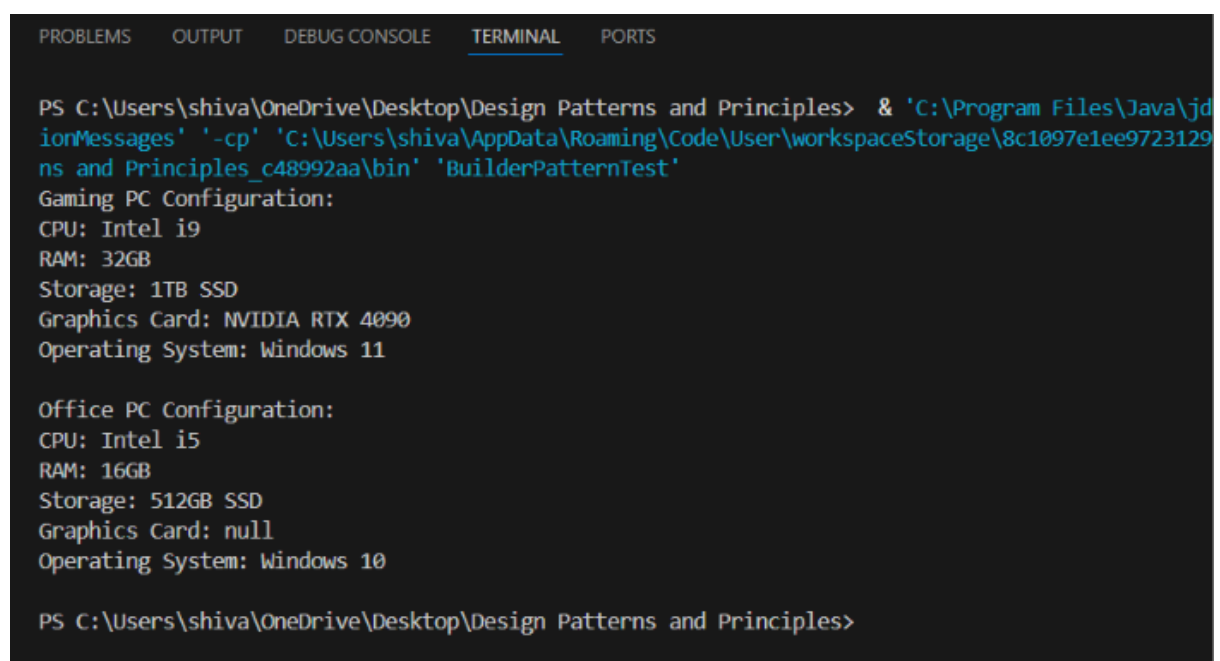
```
System.out.println("Gaming PC Configuration:");  
gamingPC.display();
```

```
System.out.println("Office PC Configuration:");  
officePC.display();
```

```
}
```

```
}
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles> & 'C:\Program Files\Java\jdk-17.0.9\bin\java.exe' -cp 'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage\8c1097e1ee9723129ns and Principles_c48992aa\bin' 'BuilderPatternTest'  
Gaming PC Configuration:  
CPU: Intel i9  
RAM: 32GB  
Storage: 1TB SSD  
Graphics Card: NVIDIA RTX 4090  
Operating System: Windows 11  
  
Office PC Configuration:  
CPU: Intel i5  
RAM: 16GB  
Storage: 512GB SSD  
Graphics Card: null  
Operating System: Windows 10  
  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles>
```

Exercise 4: Implementing the Adapter Pattern

Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **AdapterPatternExample**.
2. **Define Target Interface:**
 - Create an interface **PaymentProcessor** with methods like **processPayment()**.
3. **Implement Adaptee Classes:**
 - Create classes for different payment gateways with their own methods.
4. **Implement the Adapter Class:**
 - Create an adapter class for each payment gateway that implements **PaymentProcessor** and translates the calls to the gateway-specific methods.
5. **Test the Adapter Implementation:**
 - Create a test class to demonstrate the use of different payment gateways through the adapter.

Solution:

CODE:

```
interface PaymentProcessor {  
    void processPayment(double amount);  
}  
  
class PayPalGateway {  
  
    public void payWithPayPal(double amount) {  
        System.out.println("Payment of Rs." + amount + " processed through PayPal.");  
    }  
}  
  
class StripeGateway {
```

```
public void makePayment(double amount) {  
    System.out.println("Payment of Rs." + amount + " processed through Stripe.");  
}  
}
```

```
class RazorpayGateway {  
  
    public void sendMoney(double amount) {  
        System.out.println("Payment of Rs." + amount + " processed through Razorpay.");  
    }  
}
```

```
class PayPalAdapter implements PaymentProcessor {
```

```
    private PayPalGateway paypal;
```

```
    public PayPalAdapter(PayPalGateway paypal) {  
        this.paypal = paypal;  
    }
```

```
    @Override
```

```
    public void processPayment(double amount) {  
        paypal.payWithPayPal(amount);  
    }  
}
```

```
class StripeAdapter implements PaymentProcessor {
```

```
    private StripeGateway stripe;
```

```

public StripeAdapter(StripeGateway stripe) {
    this.stripe = stripe;
}

@Override
public void processPayment(double amount) {
    stripe.makePayment(amount);
}
}

class RazorpayAdapter implements PaymentProcessor {

    private RazorpayGateway razorpay;

    public RazorpayAdapter(RazorpayGateway razorpay) {
        this.razorpay = razorpay;
    }

    @Override
    public void processPayment(double amount) {
        razorpay.sendMoney(amount);
    }
}

public class AdapterPatternTest {

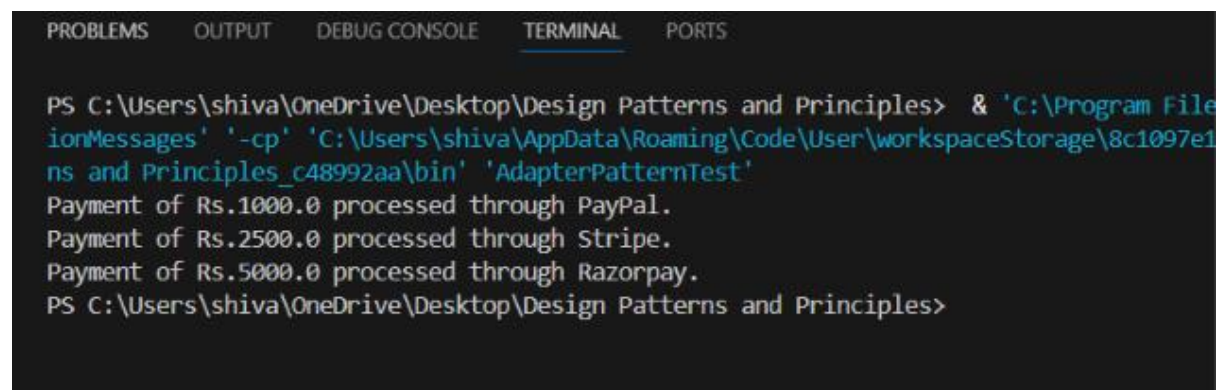
    public static void main(String[] args) {

        PaymentProcessor paypal =
            new PayPalAdapter(new PayPalGateway());
    }
}

```

```
PaymentProcessor stripe =  
    new StripeAdapter(new StripeGateway());  
  
PaymentProcessor razorpay =  
    new RazorpayAdapter(new RazorpayGateway());  
  
paypal.processPayment(1000);  
  
stripe.processPayment(2500);  
  
razorpay.processPayment(5000);  
}  
}
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles> & 'C:\Program Files  
ionMessages' '-cp' 'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage\8c1097e1  
ns and Principles_c48992aa\bin' 'AdapterPatternTest'  
Payment of Rs.1000.0 processed through PayPal.  
Payment of Rs.2500.0 processed through Stripe.  
Payment of Rs.5000.0 processed through Razorpay.  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles>
```

Exercise 5: Implementing the Decorator Pattern

Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **DecoratorPatternExample**.
2. **Define Component Interface:**
 - Create an interface **Notifier** with a method **send()**.
3. **Implement Concrete Component:**
 - Create a class **EmailNotifier** that implements **Notifier**.
4. **Implement Decorator Classes:**
 - Create abstract decorator class **NotifierDecorator** that implements **Notifier** and holds a reference to a **Notifier** object.
 - Create concrete decorator classes like **SMSNotifierDecorator**, **SlackNotifierDecorator** that extend **NotifierDecorator**.
5. **Test the Decorator Implementation:**
 - Create a test class to demonstrate sending notifications via multiple channels using decorators.

Solution:

CODE:

```
interface Notifier {
```

```
    void send(String message);
```

```
}
```

```
class EmailNotifier implements Notifier {
```

```
    @Override
```

```
    public void send(String message) {
```

```
        System.out.println("Sending Email: " + message);
```

```
    }
```

```
}
```

```
abstract class NotifierDecorator implements Notifier {
```

```
    protected Notifier notifier;
```

```
    public NotifierDecorator(Notifier notifier) {
```

```
        this.notifier = notifier;
```

```
    }
```

```
    @Override
```

```
    public void send(String message) {
```

```
        notifier.send(message);
```

```
    }
```

```
}
```

```
class SMSNotifierDecorator extends NotifierDecorator {
```

```
    public SMSNotifierDecorator(Notifier notifier) {
```

```
        super(notifier);
```

```
    }
```

```
    @Override
```

```
    public void send(String message) {
```

```
        super.send(message);
```

```
        System.out.println("Sending SMS: " + message);
```

```
    }
```

```
}
```

```
class SlackNotifierDecorator extends NotifierDecorator {
```

```

public SlackNotifierDecorator(Notifier notifier) {
    super(notifier);
}

@Override
public void send(String message) {
    super.send(message);
    System.out.println("Sending Slack Message: " + message);
}
}

public class DecoratorPatternTest {

    public static void main(String[] args) {

        System.out.println("Email Notification:");
        Notifier emailNotifier = new EmailNotifier();
        emailNotifier.send("System Update Available");

        System.out.println("\nEmail + SMS Notification:");
        Notifier smsNotifier =
            new SMSNotifierDecorator(new EmailNotifier());
        smsNotifier.send("System Update Available");

        System.out.println("\nEmail + SMS + Slack Notification:");
        Notifier allNotifier =
            new SlackNotifierDecorator(
                new SMSNotifierDecorator(
                    new EmailNotifier()));

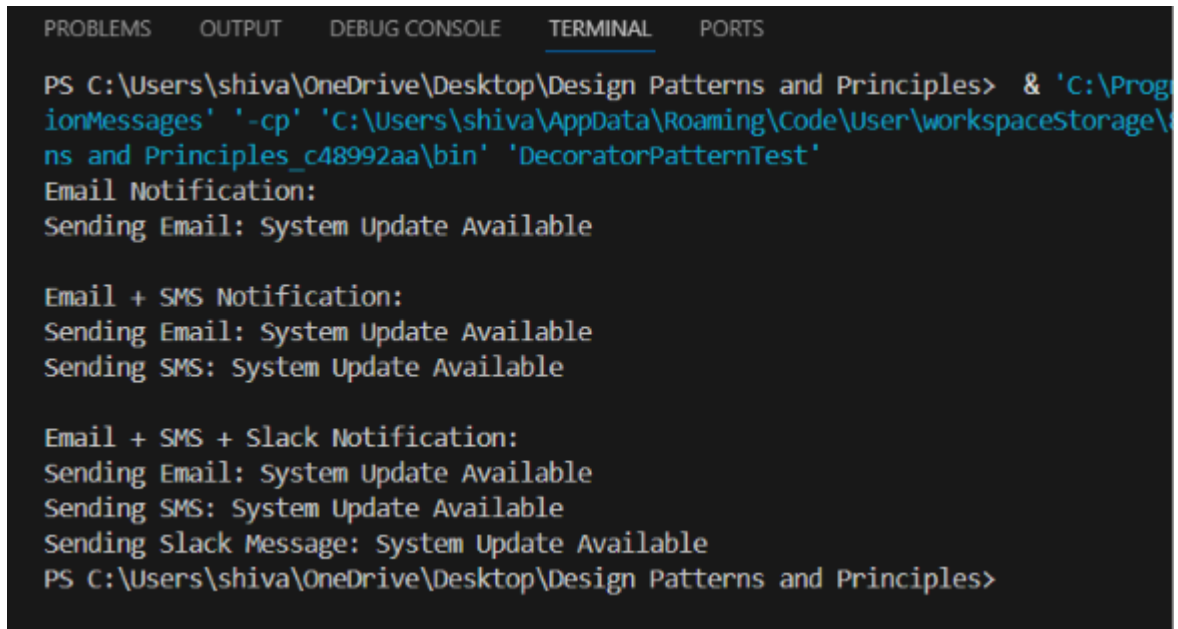
        allNotifier.send("System Update Available");
    }
}

```



```
}  
  
}
```

OUTPUT:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles> & 'C:\Program Files\Java\jdk-11.0.2\bin\java.exe' -cp 'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage\1\workspace\Design Patterns and Principles_c48992aa\bin' 'DecoratorPatternTest'  
Email Notification:  
Sending Email: System Update Available  
  
Email + SMS Notification:  
Sending Email: System Update Available  
Sending SMS: System Update Available  
  
Email + SMS + Slack Notification:  
Sending Email: System Update Available  
Sending SMS: System Update Available  
Sending Slack Message: System Update Available  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles>
```

Exercise 6: Implementing the Proxy Pattern

Scenario:

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **ProxyPatternExample**.
2. **Define Subject Interface:**
 - Create an interface **Image** with a method **display()**.
3. **Implement Real Subject Class:**
 - Create a class **RealImage** that implements **Image** and loads an image from a remote server.
4. **Implement Proxy Class:**
 - Create a class **ProxyImage** that implements **Image** and holds a reference to **RealImage**.

- Implement lazy initialization and caching in **ProxyImage**.

5. Test the Proxy Implementation:

- Create a test class to demonstrate the use of **ProxyImage** to load and display images.

Solution:

CODE:

```
interface Image {  
    void display();  
}
```

```
class ReallImage implements Image {
```

```
    private String fileName;
```

```
    public ReallImage(String fileName) {  
        this.fileName = fileName;  
        loadFromServer();  
    }
```

```
    private void loadFromServer() {  
        System.out.println("Loading image from remote server: " + fileName);  
    }
```

```
    @Override  
    public void display() {  
        System.out.println("Displaying image: " + fileName);  
    }  
}
```

```
class ProxyImage implements Image {
```

```
private String fileName;

private ReallImage reallImage;

public ProxyImage(String fileName) {
    this.fileName = fileName;
}

@Override
public void display() {

    if (reallImage == null) {
        reallImage = new ReallImage(fileName);
    }

    reallImage.display();
}
}

public class ProxyPatternTest {

    public static void main(String[] args) {

        Image image = new ProxyImage("Nature.jpg");

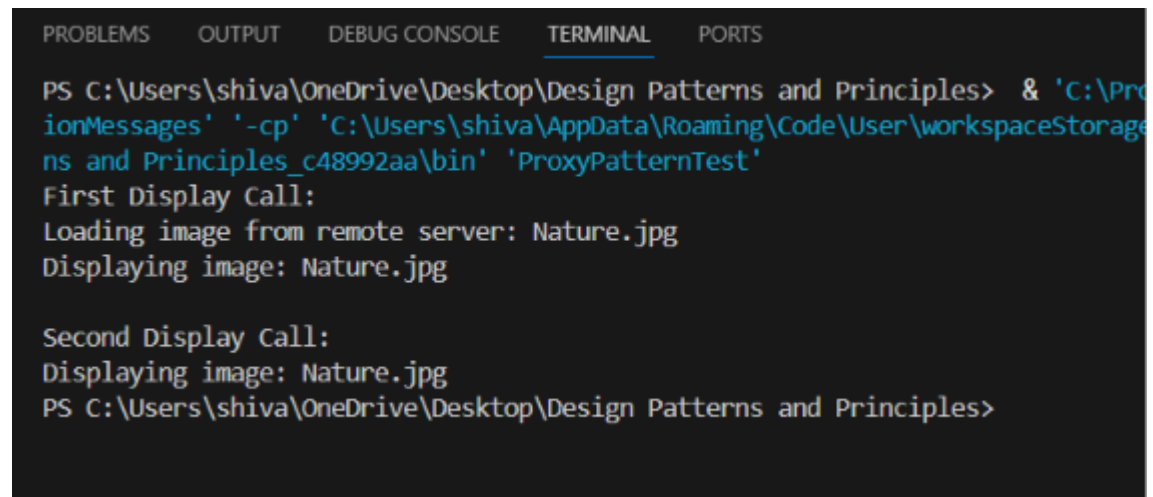
        System.out.println("First Display Call:");
        image.display();

        System.out.println();

        System.out.println("Second Display Call:");
        image.display();
    }
}
```

```
}  
}
```

OUTPUT:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles> & 'C:\Pro  
ionMessages' '-cp' 'C:\Users\shiva\AppData\Roaming\Code\User\workspaceStorage  
ns and Principles_c48992aa\bin' 'ProxyPatternTest'  
First Display Call:  
Loading image from remote server: Nature.jpg  
Displaying image: Nature.jpg  
  
Second Display Call:  
Displaying image: Nature.jpg  
PS C:\Users\shiva\OneDrive\Desktop\Design Patterns and Principles>
```